

OptiCrop

Smart Agricultural Production Optimization Engine

Project Development Phase — Document 1: Code Layout, Readability and Reusability

1. Overview

This document describes the structure, organization, and reusability principles followed during the development of the OptiCrop crop recommendation system. The project is divided into a clear separation of concerns: data analysis and model training (eda.py), the web application backend (app.py), and the presentation layer (HTML templates). This separation ensures the codebase is modular, readable, and easy to maintain or extend.

2. Project Folder Structure

The project follows a standard Flask application layout, separating code, templates, static assets, and trained model artifacts into distinct folders:

```
opticrop/
├── app.py                (Flask backend - routes & prediction logic)
├── eda.py                (Data analysis, preprocessing & model training)
├── Crop_recommendation.csv (Source dataset)
├── model.pkl             (Serialized trained Logistic Regression model)
├── scaler.pkl            (Serialized StandardScaler for input preprocessing)
├── templates/
│   ├── home.html        (Landing page)
│   ├── about.html       (Project description page)
│   └── findyourcrop.html (Prediction form & result page)
├── static/
│   └── images/
│       ├── home_bg.jpg
│       ├── about_bg.jpg
│       └── crop_bg.jpg
```

3. Code Readability Practices

The following conventions were applied consistently across the codebase to maximize readability:

- Descriptive variable names (e.g., `x_train`, `y_pred`, `sample_input`) instead of single-letter or ambiguous names, making the data flow self-explanatory.
- Section header comments (e.g., `'# ===== EPIC 4: MODEL BUILDING ====='`) clearly demarcate logical stages of the pipeline (data loading, EDA, preprocessing, model training, evaluation, deployment).
- Consistent indentation and spacing (PEP 8 style) throughout all Python files.

- Print statements with descriptive labels (e.g., 'Shape of x_train:') so console output is self-documenting during execution and debugging.
- HTML templates use semantic class names (.topnav, .content, .flask-output) rather than generic or inline styling, improving CSS readability and maintainability.

4. Reusability Design Decisions

Several design choices were made specifically to make components reusable across the analysis phase and the deployed application:

- **Model and Scaler Persistence:** The trained model and the StandardScaler are both serialized using pickle (model.pkl, scaler.pkl). This avoids retraining the model every time the application restarts, and ensures the exact same preprocessing logic (log transformation + scaling) used during training is reused identically during live predictions.
- **Shared preprocessing logic:** The same feature order (N, P, K, temperature, humidity, ph, rainfall) and the same log1p transformation on Potassium are applied in both eda.py (training) and app.py (inference), guaranteeing consistency between training and deployment.
- **Template inheritance pattern:** All three HTML pages (home, about, findyourcrop) share an identical navigation bar structure and CSS class names, allowing styling changes to be made consistently with minimal duplication.
- **Parameterized numeric_cols list:** Column names are stored once in a list (numeric_cols) and reused across multiple analysis functions (distribution plots, outlier detection, correlation heatmap) rather than being hard-coded repeatedly.
- **Modular Flask routes:** Each page (home, about, findyourcrop) and the prediction logic (predict) are implemented as separate, independent route functions, allowing each to be modified or extended without affecting the others.

5. Separation of Concerns

File	Responsibility	Reused By
eda.py	Data loading, EDA, preprocessing, model training & evaluation	Produces model.pkl, scaler.pkl used by app.py
app.py	Flask routing, request handling, calling the saved model	Loads model.pkl & scaler.pkl from eda.py
templates/*.html	User interface presentation & form input collection	Rendered by app.py routes

6. Conclusion

The OptiCrop codebase is organized into clearly separated, well-documented, and reusable components. The preprocessing pipeline built during model training is fully reused at inference time, ensuring consistency between offline analysis and the live Flask application, while the

modular file structure makes the project straightforward to maintain, test, and extend with additional features in the future.